

Bazele Inteligenței Artificiale

Lab 4. Arbori de decizie. Metode de ansambluri

Efrem Dragoș-Sebastian-Mihaly

Clasificare

- Setul de date Iris:

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.3	3.3	6.0	2.5	2
...	...	...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

Clasificare

Sepal Length	Sepal Width	Petal Length	Petal Width	Species
5.1	3.5	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.3	3.3	6.0	2.5	2
...	...	...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

0 - Iris-Setosa

1 - Iris-Versicolor

2 - Iris-Virginica

Clasificare

Sepal Length	Sepal Width	Species
5.1	3.5	0
7.0	3.2	1
6.3	3.3	2
...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

0 - Iris-Setosa

1 - Iris-Versicolor

2 - Iris-Virginica

Clasificare

Arbori de decizie

Arbori de decizie



Arbori de decizie



Cum găsim suspectul? Ne punem întrebări!

Întrebare: Suspectul purta pălărie?

- Dacă da, ne uităm acum la suspectii cu pălărie
- Dacă nu, ne uităm la cei fără pălărie

Întrebare: Suspectul era înalt?

- Dacă da, ne uităm acum la suspectii înalți
- Dacă nu, ne uităm la cei scunzi

Arbori de decizie

Sepal Length	Sepal Width	Species
5.1	3.5	0
7.0	3.2	1
6.3	3.3	2
...	...	...

Caracteristici / Attribute / Features

Etichetă / Clasă

0 - Iris-Setosa

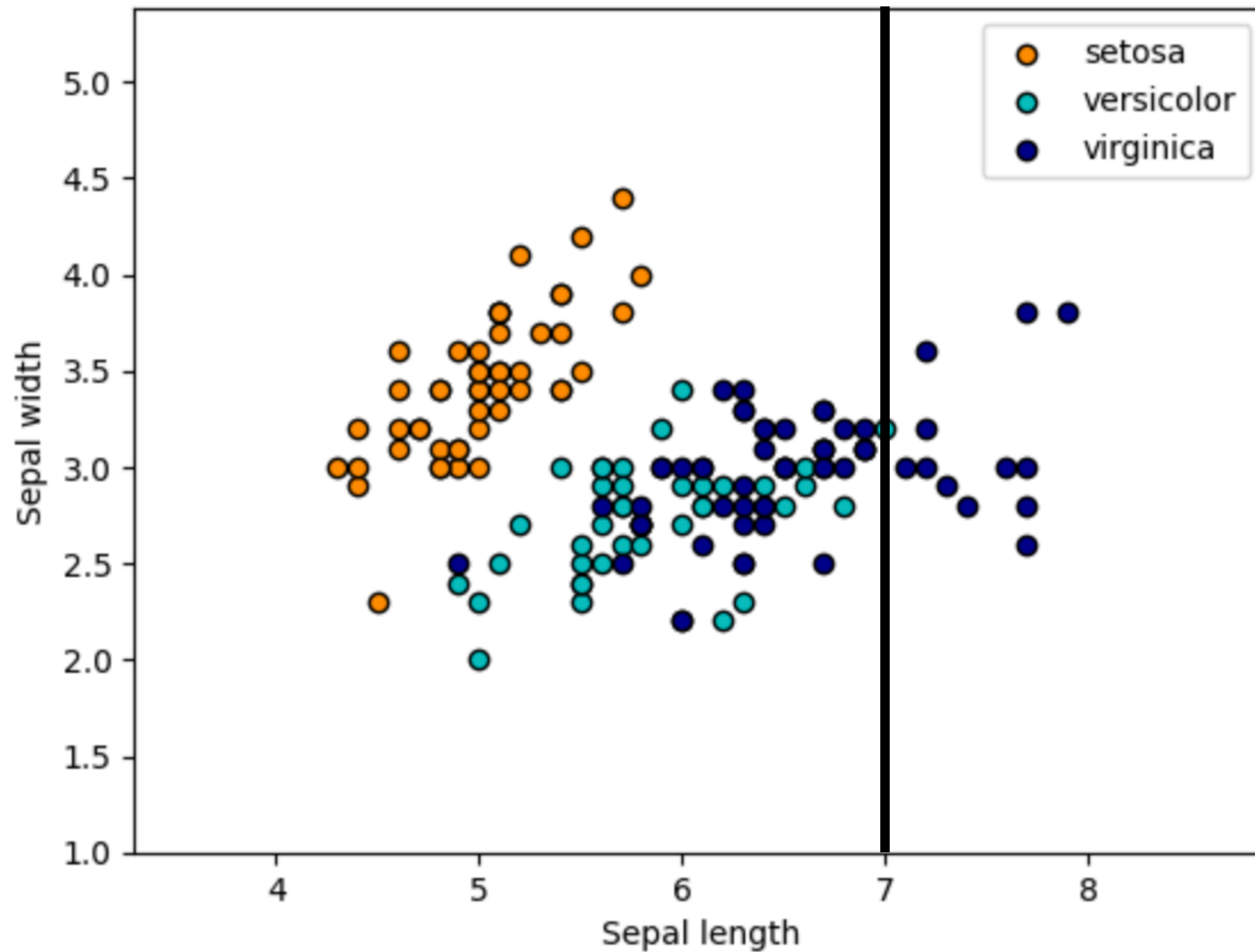
1 - Iris-Versicolor

2 - Iris-Virginica

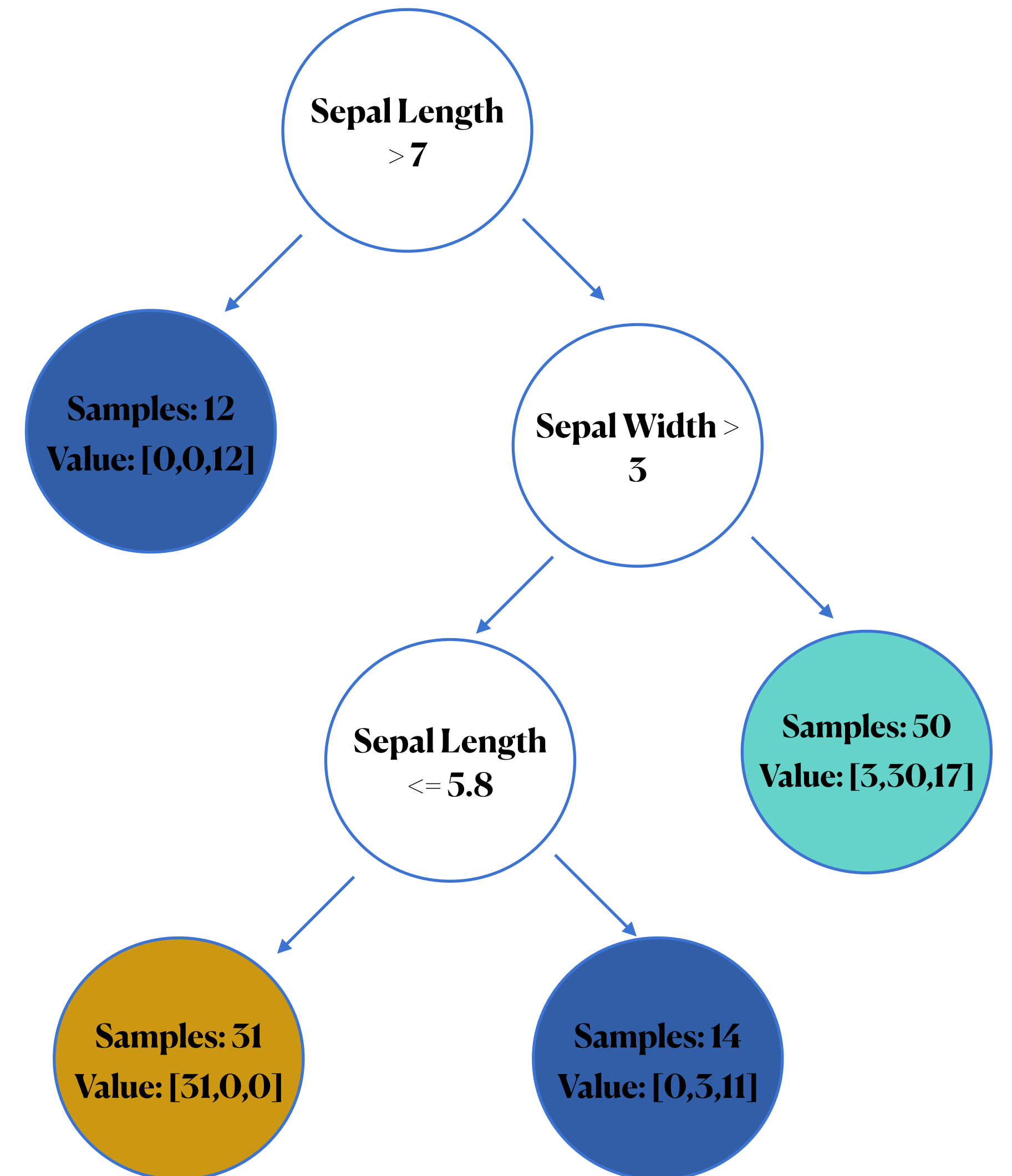
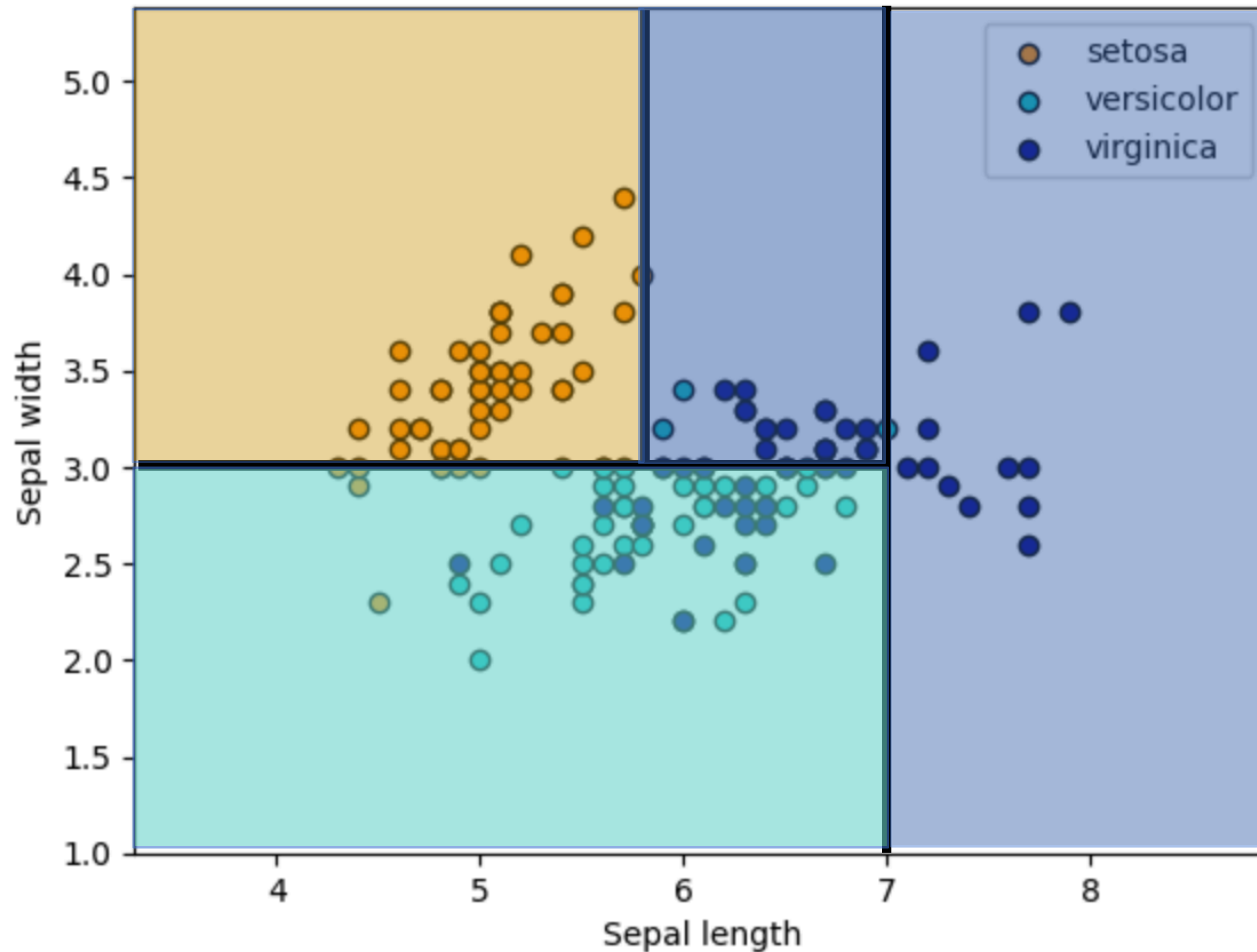
Arbori de decizie

- Arborii de decizie pun întrebări cu privire la caracteristicile setului de antrenare.
- Scopul este de a împărți setul de date în diverse regiuni unde clasificarea se poate realiza mai ușor.
- Rezultatul este un arbore binar care ajută la a oferi etichete noi la date nemaivăzute.

Arbori de decizie



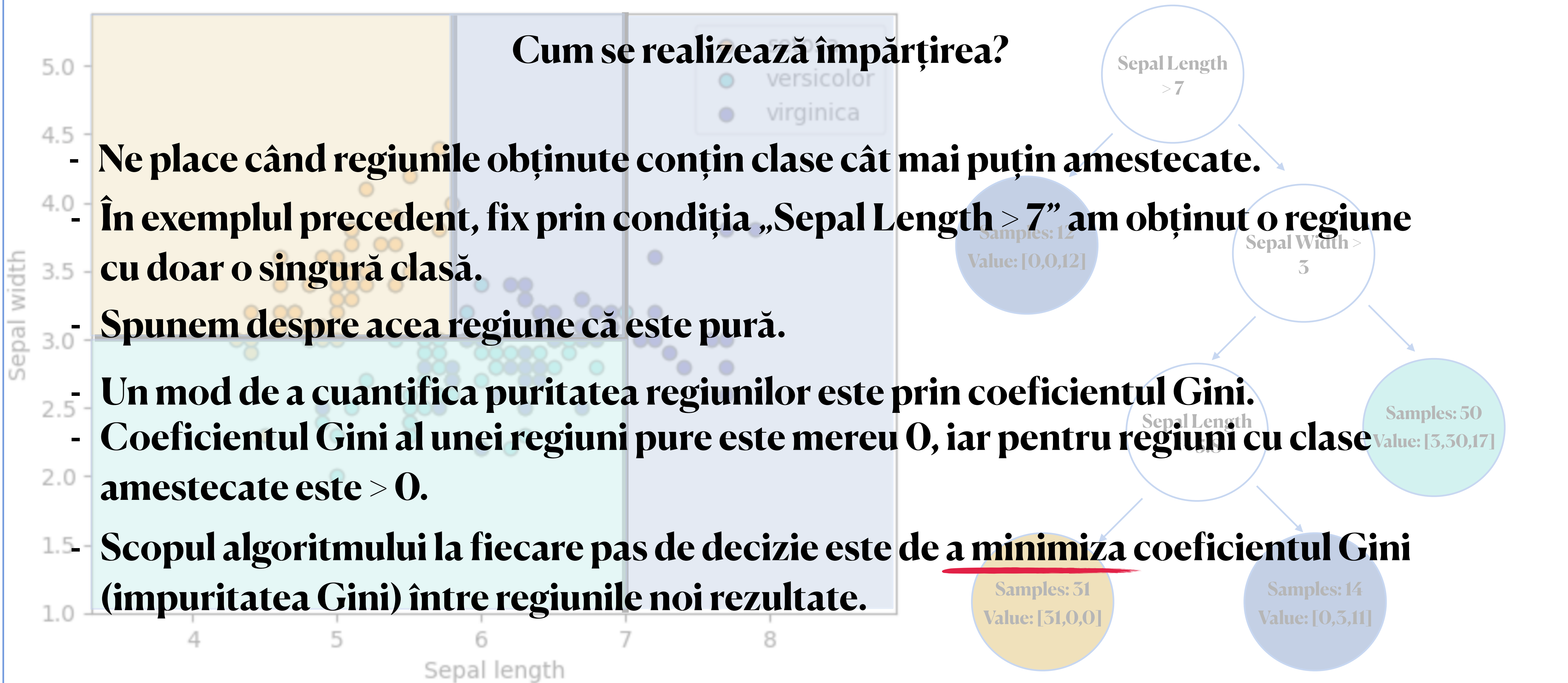
Arbori de decizie



Arbori de decizie

Cum se realizează împărțirea?

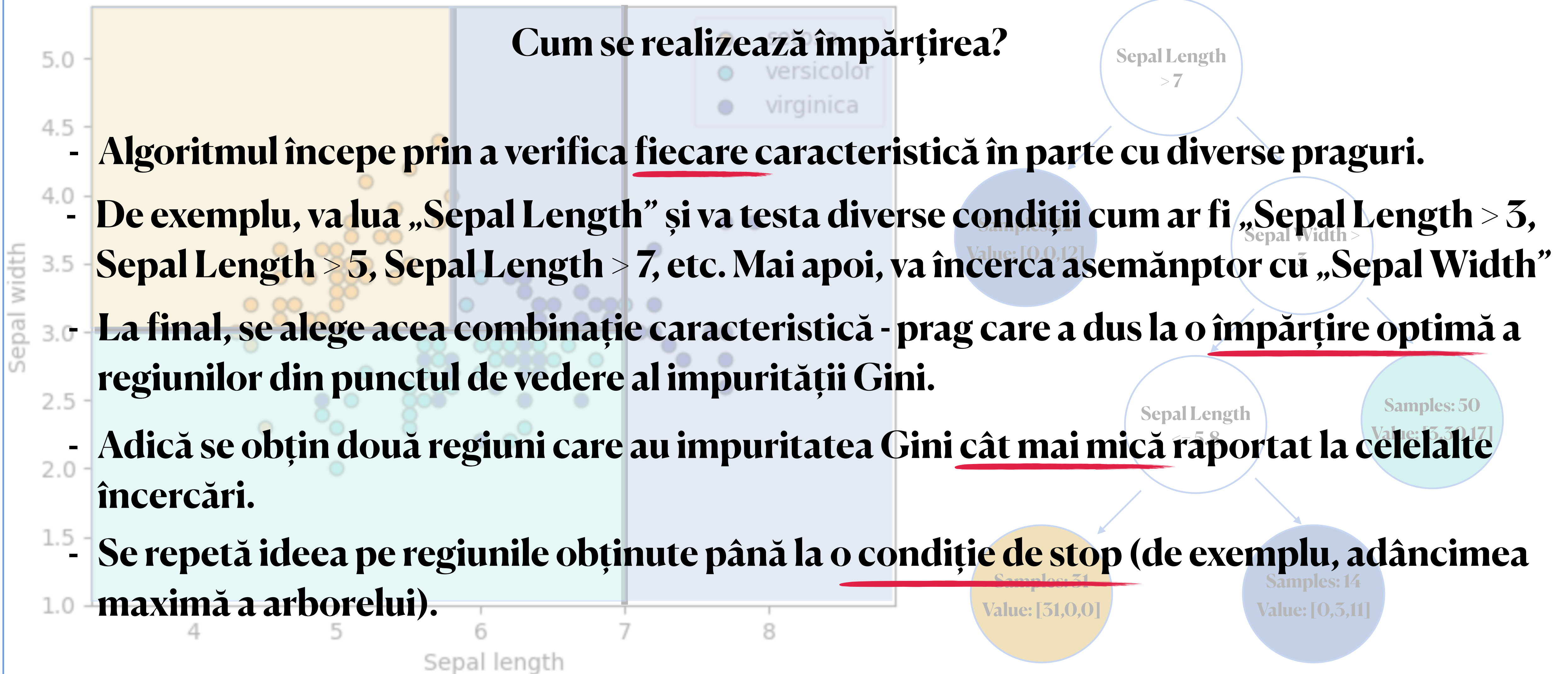
- Ne place când regiunile obținute conțin clase cât mai puțin amestecate.
- În exemplul precedent, fix prin condiția „Sepal Length > 7 ” am obținut o regiune cu doar o singură clasă.
- Spunem despre acea regiune că este pură.
- Un mod de a cuantifica puritatea regiunilor este prin coeficientul Gini.
- Coeficientul Gini al unei regiuni pure este mereu 0, iar pentru regiuni cu clase amestecate este > 0 .
- Scopul algoritmului la fiecare pas de decizie este de a minimiza coeficientul Gini (impuritatea Gini) între regiunile noi rezultate.



Arbori de decizie

Cum se realizează împărțirea?

- Algoritmul începe prin a verifica fiecare caracteristică în parte cu diverse praguri.
- De exemplu, va lua „Sepal Length” și va testa diverse condiții cum ar fi „Sepal Length > 3, Sepal Length > 5, Sepal Length > 7, etc. Mai apoi, va încerca asemănător cu „Sepal Width”
- La final, se alege acea combinație caracteristică - prag care a dus la o împărțire optimă a regiunilor din punctul de vedere al impurității Gini.
- Adică se obțin două regiuni care au impuritatea Gini cât mai mică raportat la celelalte încercări.
- Se repetă ideea pe regiunile obținute până la o condiție de stop (de exemplu, adâncimea maximă a arborelui).



Arbori de decizie

```
▶ import math
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris_dataset = load_iris()
X = iris_dataset.data[:, :2] # sepal length and width
y = iris_dataset.target

d = 2 # numarul de features

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
tree_clf = DecisionTreeClassifier(max_depth = 10, min_samples_leaf = 3, max_features=int(math.sqrt(d)), random_state=42)

tree_clf.fit(X_train, y_train)

y_pred = tree_clf.predict(X_test)

tree_clf.feature_importances_, accuracy_score(y_test, y_pred), tree_clf.predict_proba([X_test[4]])

➡ (array([0.69076863, 0.30923137]), 0.7368421052631579, array([[0. , 0.4, 0.6]]))
```

Arbori de decizie

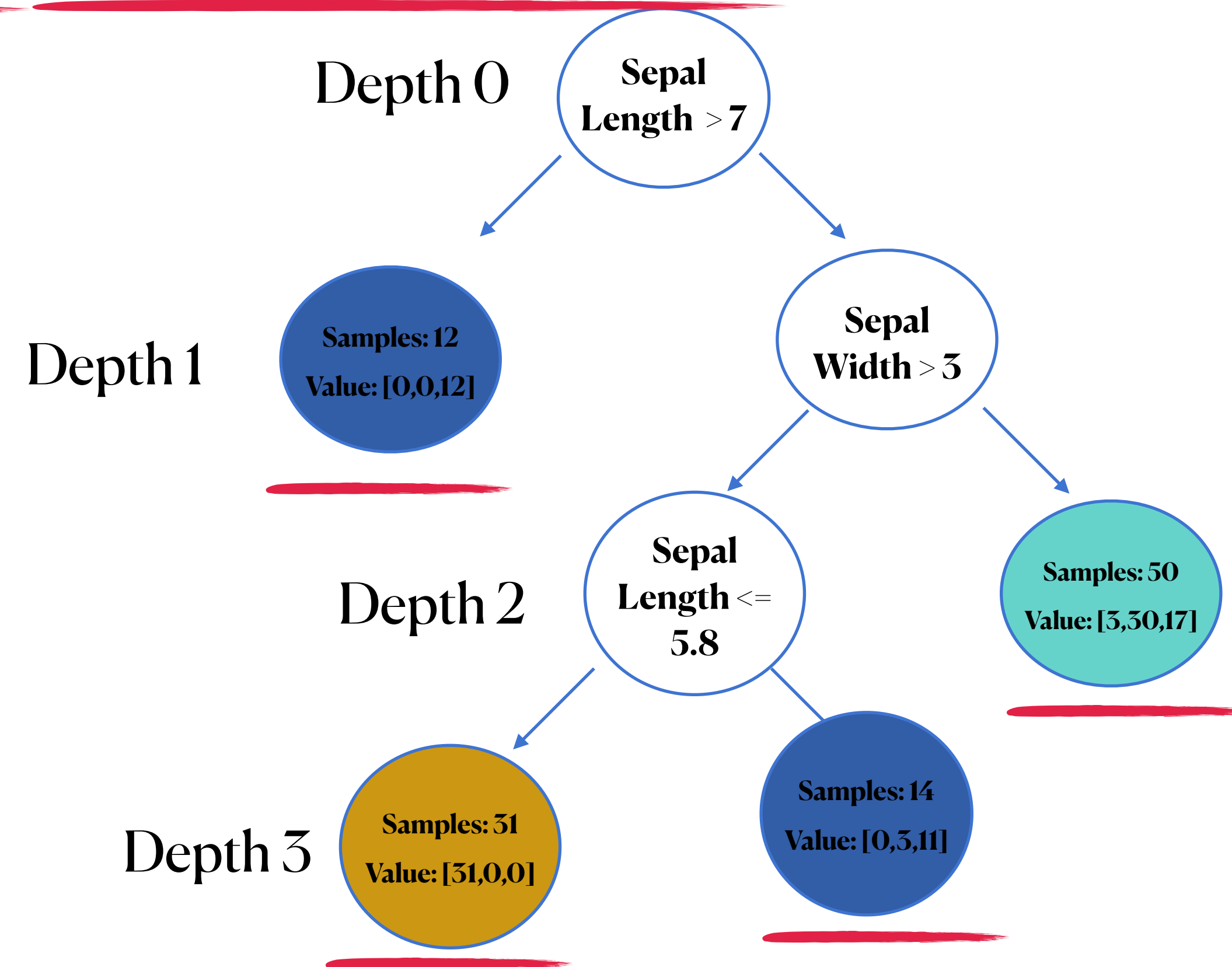
```
tree_clf = DecisionTreeClassifier(max_depth = 10, min_samples_leaf = 3, max_features=int(math.sqrt(d)), random_state=42)
```

max_depth se referă la adâncimea maximă pe care o poate avea arborele.

min_samples_leaf se referă la numărul minim de puncte pe care o regiune / frunză trebuie să le aibă.

max_features se referă la numărul maxim de caracteristici pe care algoritmul le folosește pentru a lua o decizie la un moment dat.

max_features este de preferat să aibă valoarea $\sqrt{d}$ unde d reprezintă numărul de features, dar depinde de problemă și cerințe.



Arbori de decizie

```
▶ import math
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris_dataset = load_iris()
X = iris_dataset.data[:, :2] # sepal length and width
y = iris_dataset.target

d = 2 # numarul de features

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
tree_clf = DecisionTreeClassifier(max_depth = 10, min_samples_leaf = 3, max_features=int(math.sqrt(d)), random_state=42)

tree_clf.fit(X_train, y_train)

y_pred = tree_clf.predict(X_test)

tree_clf.feature_importances_, accuracy_score(y_test, y_pred), tree_clf.predict_proba([X_test[4]])
```

⇒ (array([0.69076863, 0.30923137]), 0.7368421052631579, array([[0. , 0.4, 0.6]]))

cât de important e fiecare feature

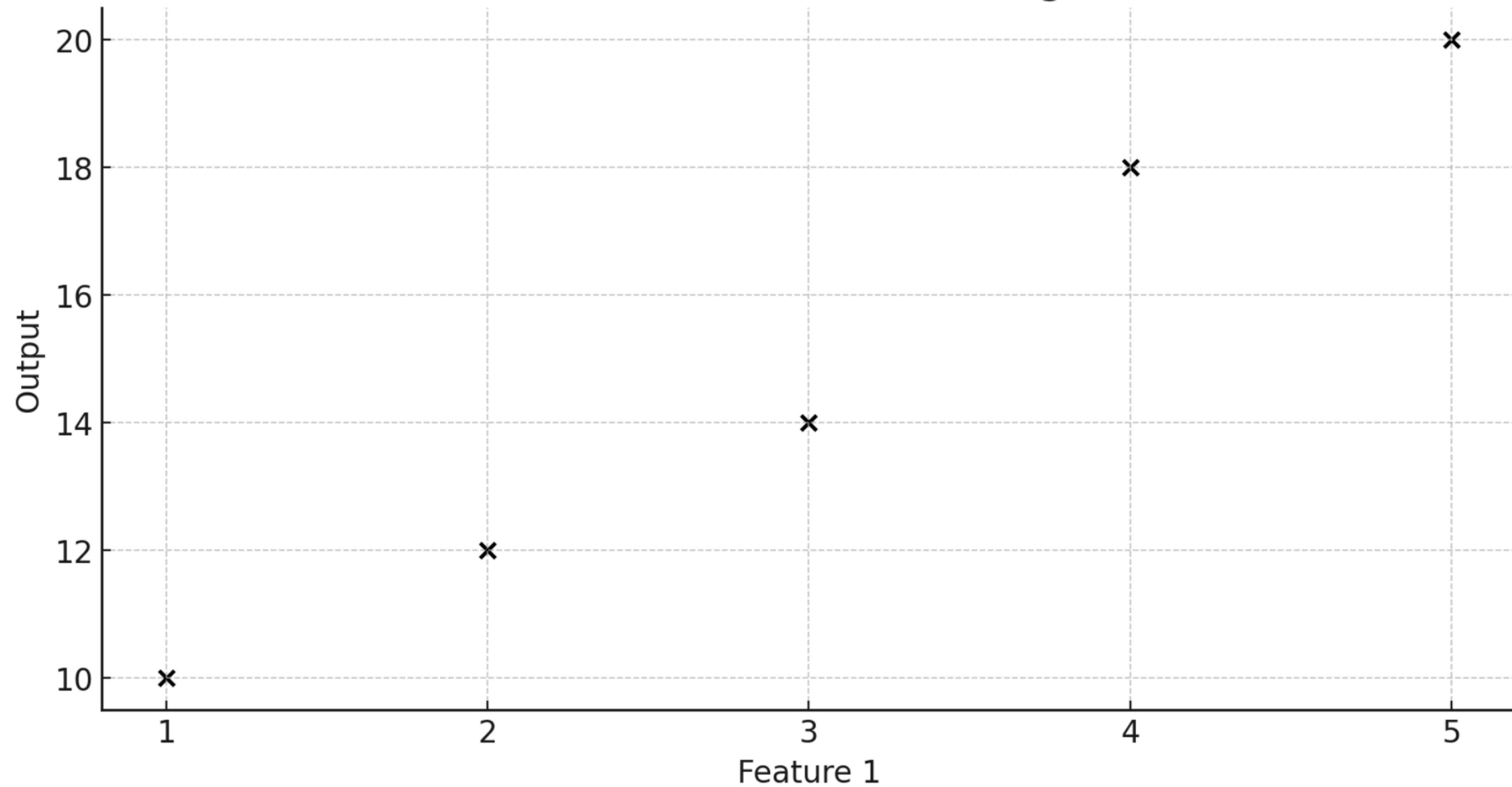
acuratețea

predicția pe al 5-lea exemplar din setul de test

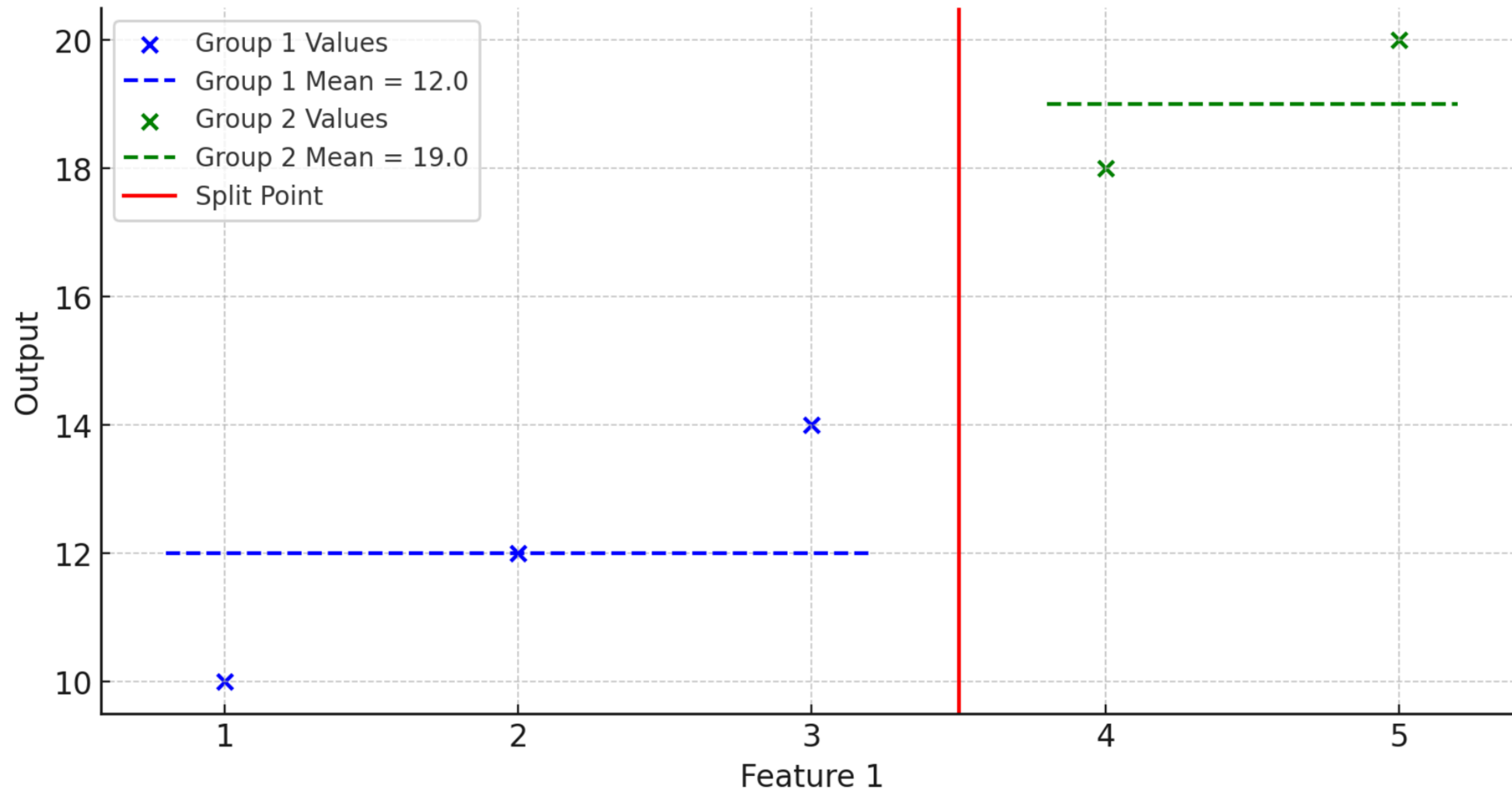
Arbori de decizie

- Pe lângă clasificare, arborii de decizie funcționează și la regresie.
- Diferența stă în două aspecte:
 - valoarea prezisă într-o regiune este media valorilor punctelor din acea regiune.
 - se minimizează eroarea medie pătratică în loc de impuritatea gini;
 - eroarea pătratică se calculează între toate punctele și valoarea medie, în final realizându-se o medie a erorilor pentru a obține eroare medie pe o anumită regiune.

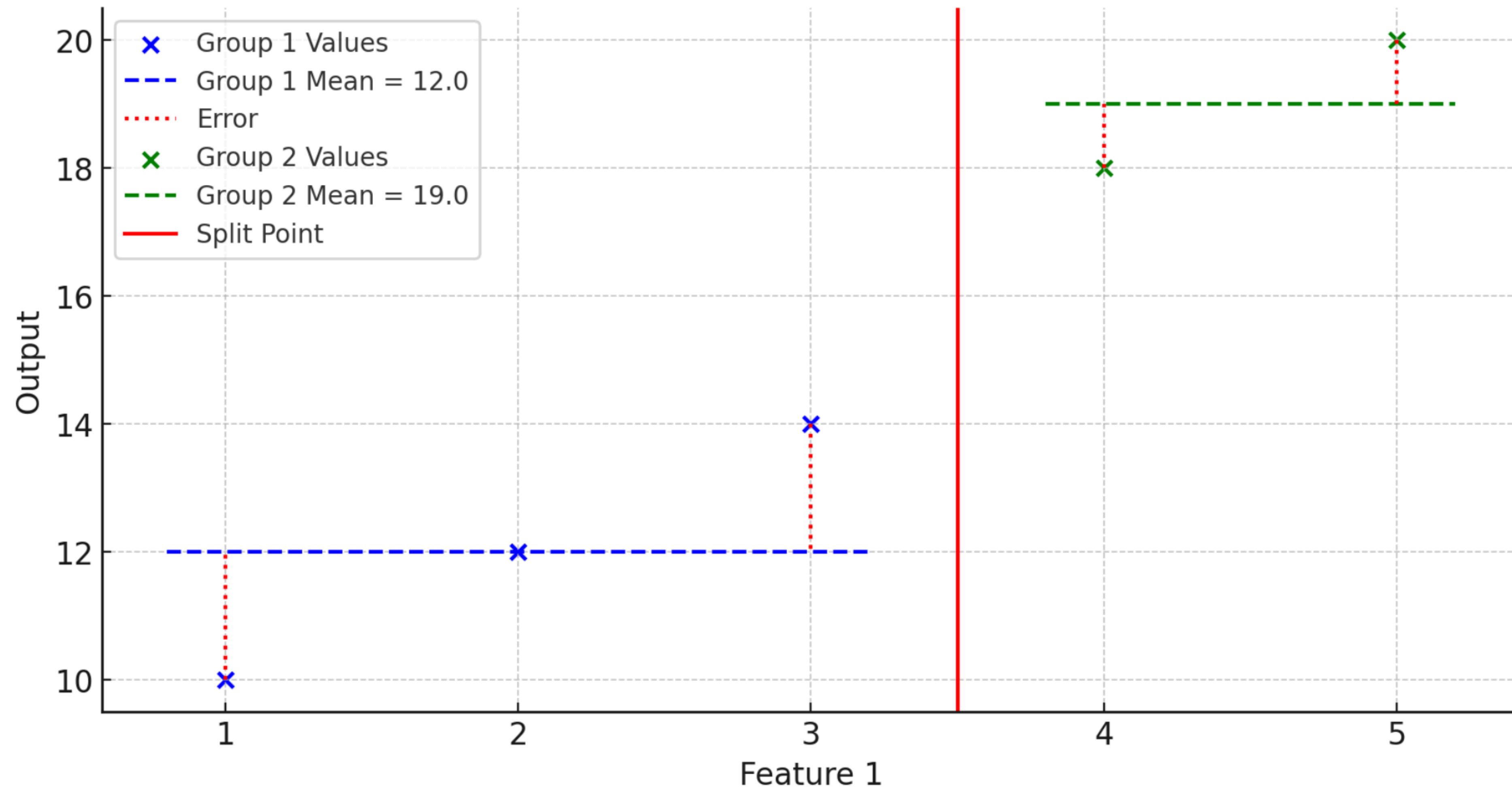
Arbori de decizie



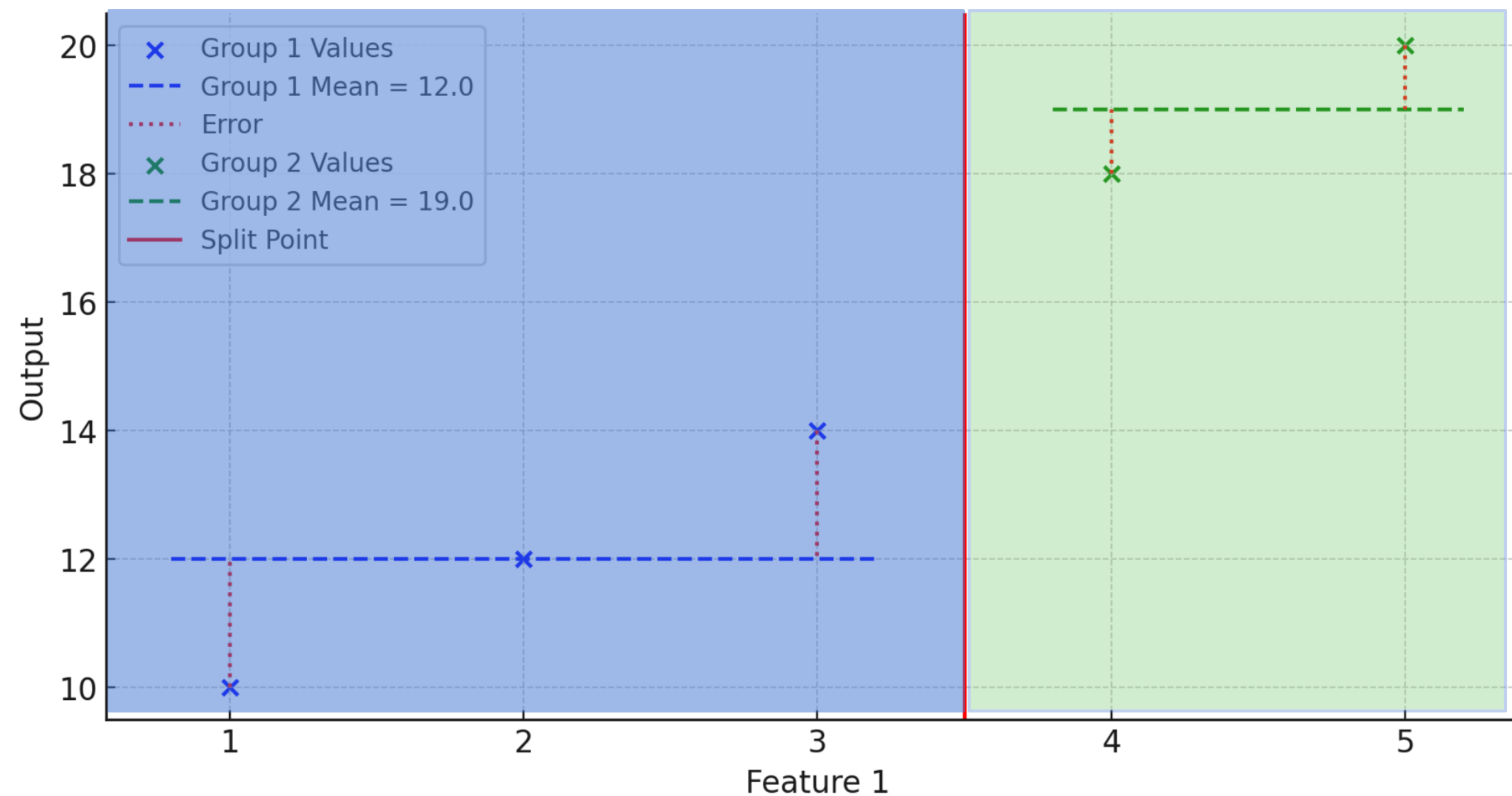
Arbori de decizie



Arbori de decizie



Arbori de decizie



Orice punct în regiunea **albastră** va fi prezis cu valoarea 12.

Orice punct în regiunea **verde** va fi prezis cu valoarea 19.

Arbori de decizie

```
import math
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

np.random.seed(42)
X = np.random.rand(150, 2) # Dataset random cu 2 features
y = np.random.rand(150)

d = 2 # number caracteristici

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

tree_reg = DecisionTreeRegressor(max_depth=10, min_samples_leaf=3, max_features=int(math.sqrt(d)), random_state=42)
tree_reg.fit(X_train, y_train)

y_pred = tree_reg.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

tree_reg.feature_importances_, mse, tree_reg.predict([X_test[4]])
```

(array([0.63367484, 0.36632516]), 0.12341939080535436, array([0.80835454]))

Idee

Ce ar fi dacă, pentru un anumit tip de problemă (fie regresie fie clasificare), nu folosim doar un singur tip model, ci mai multe tipuri de modele?

Și să agregăm rezultatele de la ele cumva

Regresie:

- regresia liniară**
- regresia polinomială**
- regresia ridge, lasso, elastic net**
- SVM**
- Arbori de decizie**

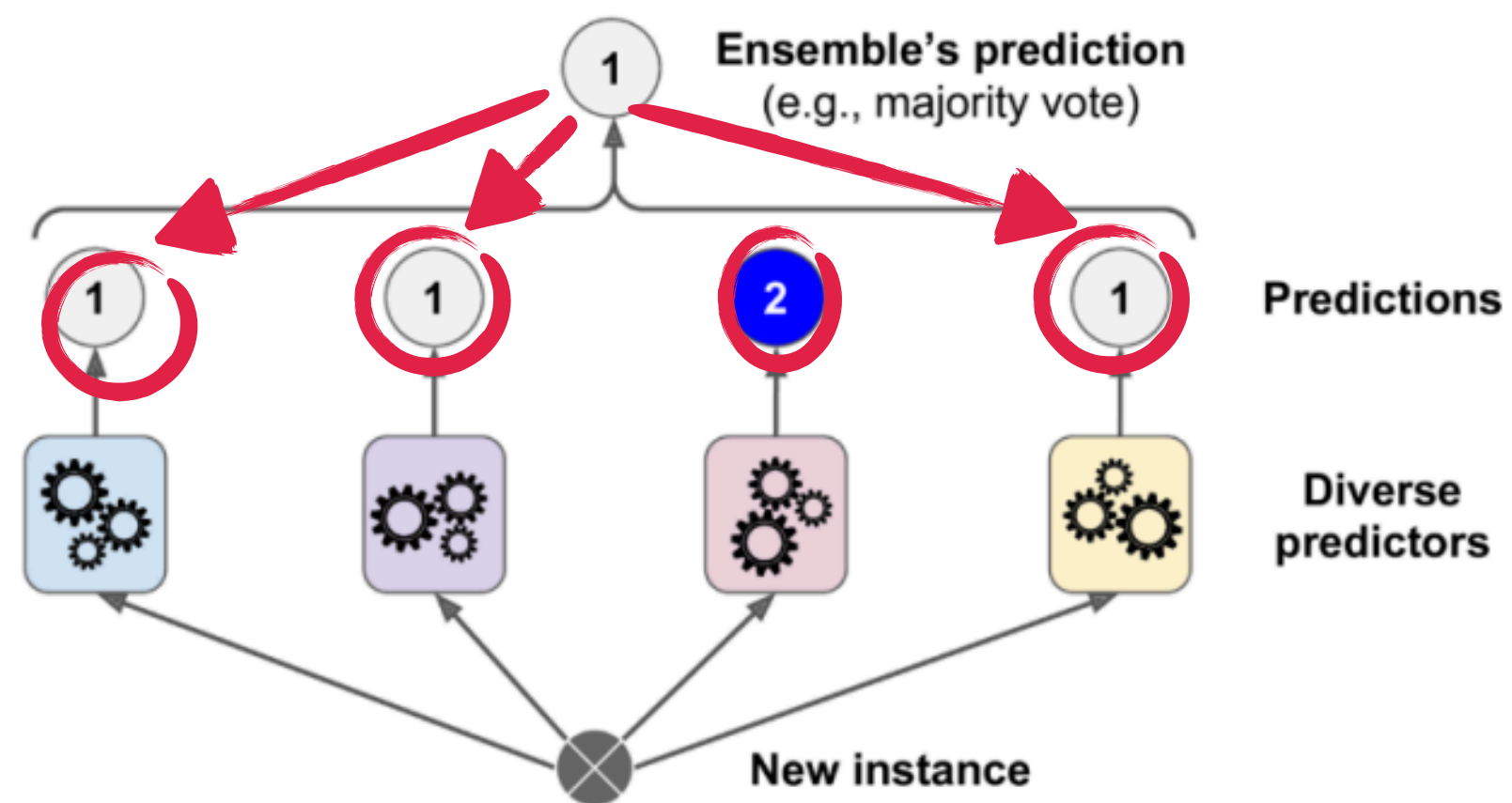
Clasificare:

- regresia logistică**
- regresia softmax**
- K-Nearest Neighbours**
- SVM**
- Arbori de decizie**

Voting

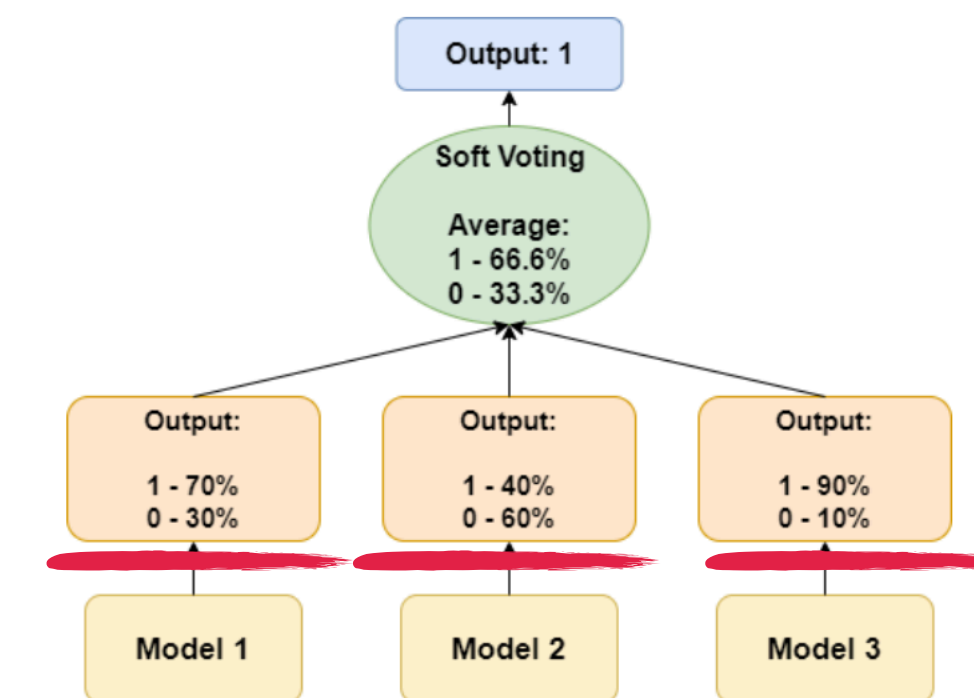
Voting (la clasificare)

Hard Voting



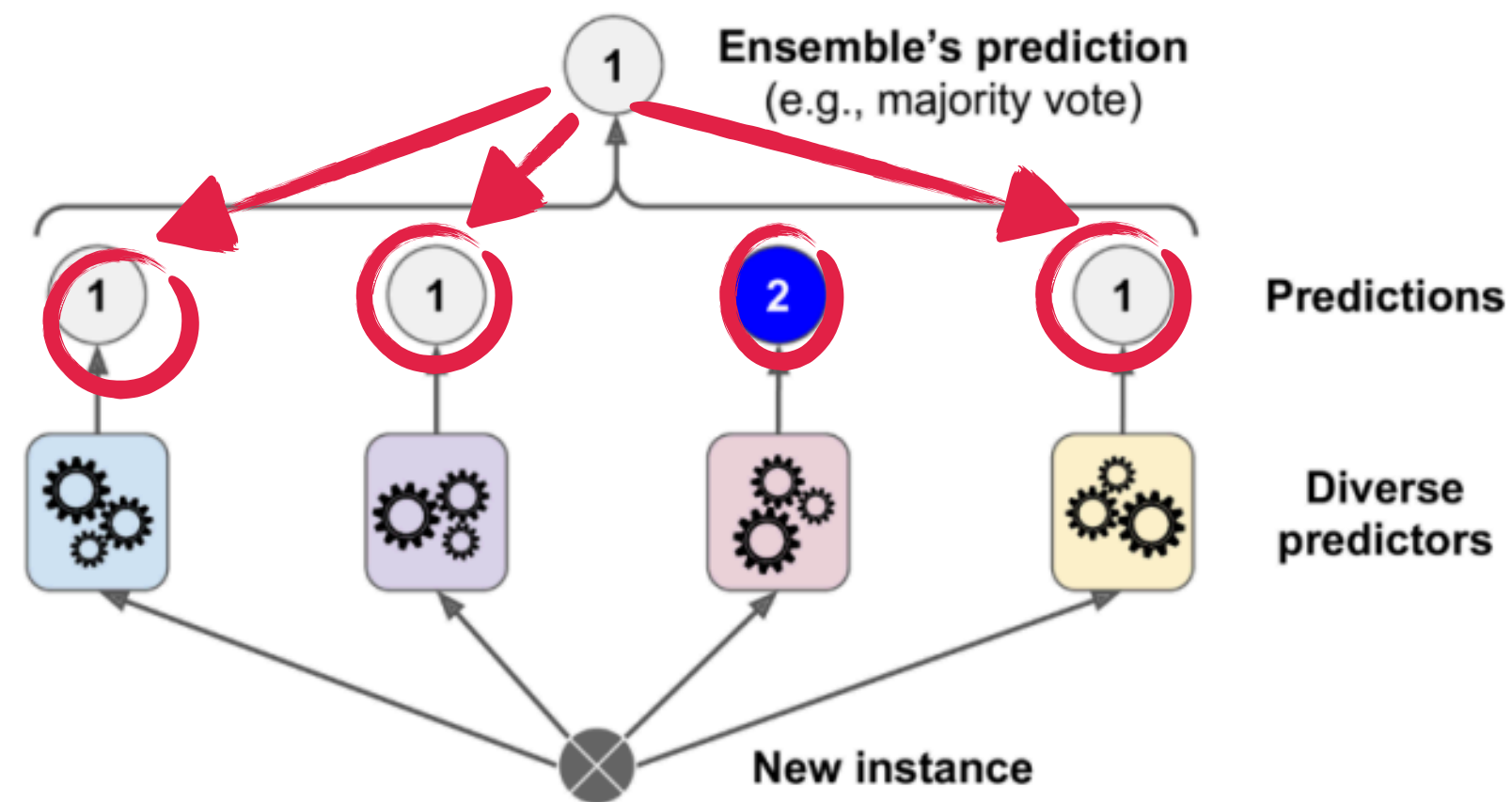
- fiecare model din ansamblu (un grup de modele) face propria predicție (ca un “vot”).
- predicția finală se bazează pe votul majoritar. Eticheta/clasa care primește cele mai multe voturi câștigă.

Soft Voting



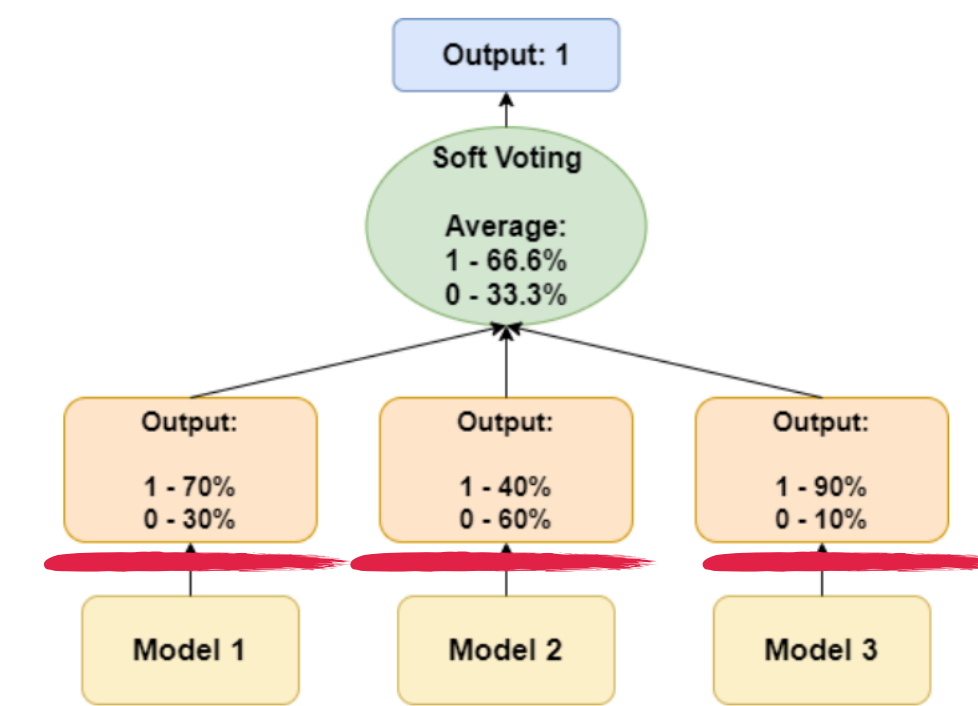
- în loc să voteze doar pentru o etichetă, fiecare model oferă probabilități pentru fiecare clasă.
- predicția finală se face prin medierea probabilităților de la toate modelele. Clasa cu probabilitatea medie cea mai mare este aleasă.

Hard Voting



- fiecare model din ansamblu (un grup de modele) face propria predicție (ca un “vot”).
- predicția finală se bazează pe votul majoritar. Eticheta/clasa care primește cele mai multe voturi câștigă.

Soft Voting



- în loc să voteze doar pentru o etichetă, fiecare model oferă probabilități pentru fiecare clasă.
- predicția finală se face prin medierea probabilităților de la toate modelele. Clasa cu probabilitatea medie cea mai mare este aleasă.

Hard voting se bazează pe numărarea voturilor, deci este mai simplu și poate ignora cât de “sigur” este un model în decizia sa.

Soft voting ia în calcul probabilitățile și poate genera rezultate mai precise când modelele sunt încrezătoare pe diferite clase.

Voting (la clasificare)

Hard Voting

```
X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
svm_clf = SVC(gamma="scale", random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='hard')

voting_clf.fit(X_train, y_train)

# Accuracy on the test set
for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.912
```

Soft Voting

```
log_clf = LogisticRegression(solver="lbfgs", random_state=42)
rnd_clf = RandomForestClassifier(n_estimators=100, random_state=42)
svm_clf = SVC(gamma="scale", probability=True, random_state=42)

voting_clf = VotingClassifier(
    estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)],
    voting='soft')
voting_clf.fit(X_train, y_train)

for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.92
```

Voting (la regresie)

Hard Voting

```
from sklearn.ensemble import VotingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import numpy as np

np.random.seed(42)
X = np.random.rand(1000, 3)
y = 3 * X[:, 0] + 5 * X[:, 1] + np.random.rand(1000)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

tree = DecisionTreeRegressor(max_depth=5, random_state=42)
svr = SVR(kernel="rbf", C=10, epsilon=0.8)

voting_reg = VotingRegressor(estimators=[('tree', tree), ('svr', svr)])

voting_reg.fit(X_train, y_train)
y_pred = voting_reg.predict(X_test)

for reg in (tree, svr, voting_reg):
    reg.fit(X_train, y_train)
    y_pred_individual = reg.predict(X_test)
    print(f"{reg.__class__.__name__} MSE: {mean_squared_error(y_test, y_pred_individual)}")
```

```
DecisionTreeRegressor MSE: 0.16732133654761192
SVR MSE: 0.13545095961917525
VotingRegressor MSE: 0.11860022682601351
```


- **Am reușit să combinăm mai multe tipuri de modele pentru a crește acuratețea generală**
- **Toate modele folosite în procesul de votare din slide-ul precedent au fost antrenate pe același set de date**
- **Ce ar fi dacă fiecare model din procesul de votare este antrenat pe un subset diferit al setului de date?**
- **Vom forța astfel ca fiecare model să vadă o realitate diferită asupra setului de date**
- **Această diversificare duce la o generalizare mai bună și combatem overfitting-ul**

Bagging și pasting

Bagging și pasting (la clasificare)

```
bag_clf = BaggingClassifier(  
    DecisionTreeClassifier(), n_estimators=500,  
    max_samples=100, bootstrap=True, random_state=42)  
bag_clf.fit(X_train, y_train)  
y_pred = bag_clf.predict(X_test)  
  
print("Bagging classifier: ", accuracy_score(y_test, y_pred))  
  
tree_clf = DecisionTreeClassifier(random_state=42)  
tree_clf.fit(X_train, y_train)  
y_pred_tree = tree_clf.predict(X_test)  
print("One decision tree classifier: ", accuracy_score(y_test, y_pred_tree))
```

Bagging classifier: 0.904

One decision tree classifier: 0.856

Bagging și pasting (la clasificare)

```
bag_clf = BaggingClassifier(  
    DecisionTreeClassifier(), n_estimators=500,  
    max_samples=100, bootstrap=True, random_state=42)  
bag_clf.fit(X_train, y_train)  
y_pred = bag_clf.predict(X_test)  
  
print("Bagging classifier: ", accuracy_score(y_test, y_pred))  
  
tree_clf = DecisionTreeClassifier(random_state=42)  
tree_clf.fit(X_train, y_train)  
y_pred_tree = tree_clf.predict(X_test)  
print("One decision tree classifier: ", accuracy_score(y_test, y_pred_tree))
```

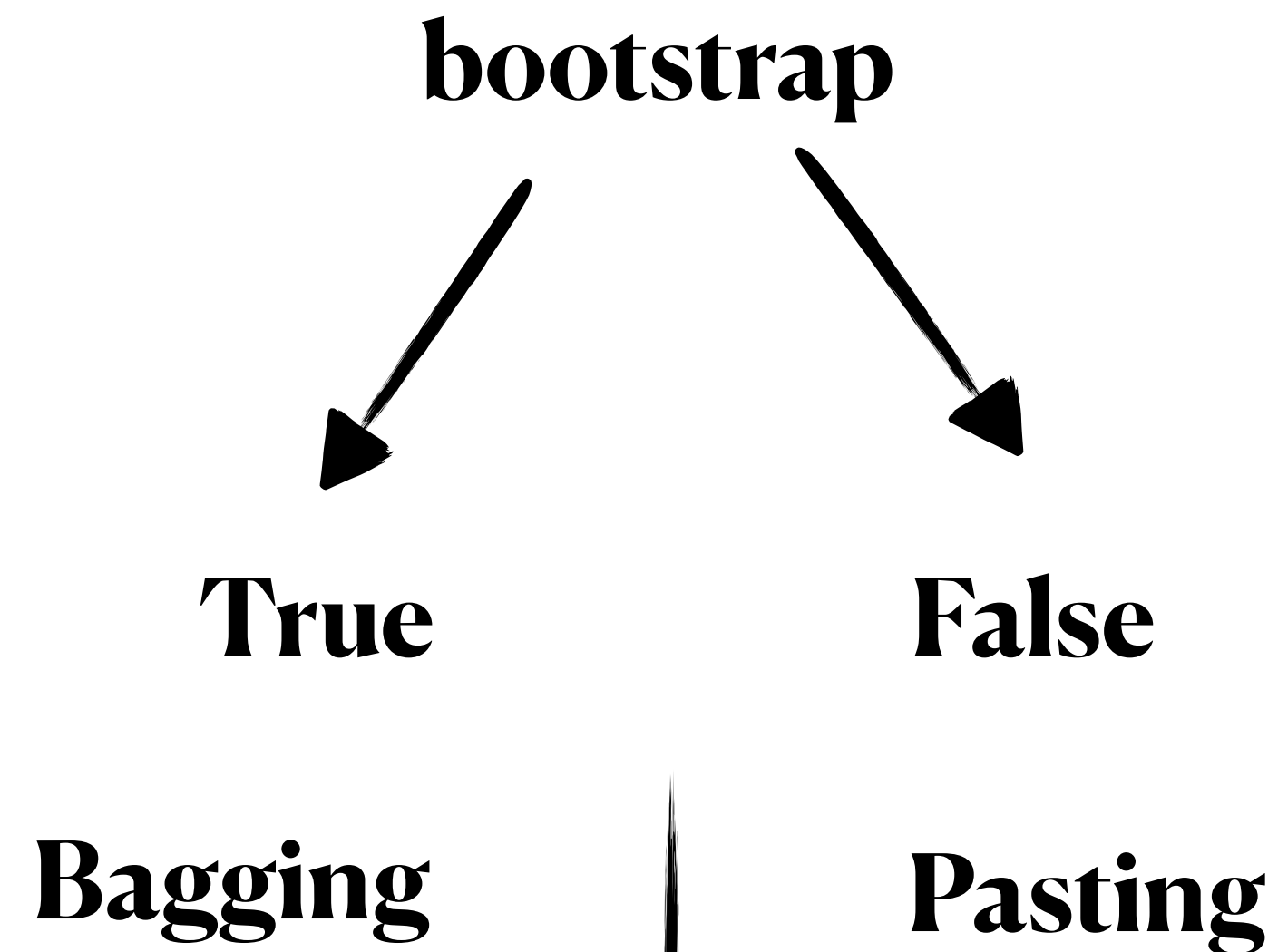
```
Bagging classifier:  0.904  
One decision tree classifier:  0.856
```

Parametri:

- **modelul folosit în a realiza ansamblul**
- **numărul de modele din ansamblu**
- **numărul maxim de exemplare cu care se antrenează fiecare model**

bootstrap

Bagging și pasting (la clasificare)



- Fiecare estimator este antrenat pe un set de date care poate conține duplicate;
- Seturile de date de la un estimator la altul se pot suprapune.

- Fiecare estimator este antrenat pe un set de date care poate conține doar elemente unice;
- Seturile de date de la un estimator la altul nu se pot suprapune.

Bagging și pasting (la clasificare)

Bagging

- Fiecare estimator este antrenat pe un set de date care poate conține duplicate;
- Seturile de date de la un estimator la altul se pot suprapune.

Pasting

- Fiecare estimator este antrenat pe un set de date care poate conține doar elemente unice;
- Seturile de date de la un estimator la altul nu se pot suprapune.

Exemplu: avem un set de date cu următoarele puncte [A,B,C,D,E] și 2 estimatori

Estimatorul 1 ar putea primi: [A,B,A,D,E]

Estimatorul 2 ar putea primi: [B,C,D,B,E]

Estimatorul 1 ar putea primi: [A,B,D]

Estimatorul 2 ar putea primi: [C,E]

Bagging și pasting (la clasificare)

```
bag_clf = BaggingClassifier(  
    DecisionTreeClassifier(), n_estimators=500,  
    max_samples=100, bootstrap=True, random_state=42)  
bag_clf.fit(X_train, y_train)  
y_pred = bag_clf.predict(X_test)  
  
print("Bagging classifier: ", accuracy_score(y_test, y_pred))  
  
tree_clf = DecisionTreeClassifier(random_state=42)  
tree_clf.fit(X_train, y_train)  
y_pred_tree = tree_clf.predict(X_test)  
print("One decision tree classifier: ", accuracy_score(y_test, y_pred_tree))
```

```
Bagging classifier:  0.904  
One decision tree classifier:  0.856
```

Parametri:

- **modelul folosit în a realiza ansamblul**
- **numărul de modele din ansamblu**
- **numărul maxim de exemplare cu care se antrenează fiecare model**

bootstrap

Bagging și pasting (la regresie)

```
from sklearn.ensemble import BaggingRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

np.random.seed(42)
X = np.random.rand(1000, 2)
y = 3 * X[:, 0] + 5 * X[:, 1] + np.random.rand(1000)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

bag_reg = BaggingRegressor(
    DecisionTreeRegressor(),
    n_estimators=500,
    max_samples=100,
    bootstrap=True,
    random_state=42
)
bag_reg.fit(X_train, y_train)
y_pred_bag = bag_reg.predict(X_test)

tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(X_train, y_train)
y_pred_tree = tree_reg.predict(X_test)

mse_bag = mean_squared_error(y_test, y_pred_bag)
mse_tree = mean_squared_error(y_test, y_pred_tree)

mse_tree, mse_bag
```

➡ (0.15320172981045363, 0.0994870232170673)

Arbori de decizie

Voting

Bagging / Pasting

+

X

=

Random Forests

Random Forests

- În Random Forests, creăm o mulțime de arbori de decizie, similar cu ceea ce am discutat la bagging și pasting. Fiecare arbore este antrenat pe un subset aleatoriu diferit de date (exact cum am făcut cu bagging sau pasting).
- Când fiecare arbore este construit, în loc să se uite la toate caracteristicile la fiecare punct de decizie, **X** alegem aleatoriu doar câteva caracteristici pentru fiecare împărțire. Acest lucru face arborii și mai diverși, deoarece nu se împart pe aceleași caracteristici.
- Odată ce toți arborii sunt construiți, combinăm rezultatele lor. Dacă facem clasificare, cerem fiecărui arbore „votul” său pentru eticheta de clasă, iar clasa cu cele mai multe voturi câștigă (exact ca la votare). Pentru regresie, facem media predicțiilor din toți arborii.
- Random Forests sunt grozave pentru că aleatoritatea în date (bagging/pasting) și aleatoritatea în caracteristici ajută modelul să evite overfitting-ul. Iar combinând rezultatele tuturor arborilor, obținem o predicție mai stabilă și mai precisă.

Random Forests

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf_clf = RandomForestClassifier(
    n_estimators=100,      # Number of trees in the forest
    max_depth=5,          # Maximum depth of each tree
    max_leaf_nodes=15,     # Maximum leaf nodes each tree can have
    min_samples_leaf=2,    # Minimum samples required at each leaf node
    random_state=42,       # Ensures reproducibility
    max_features='sqrt',   # Number of features to consider for splits
    bootstrap=True         # Whether to use bagging (true) or pasting (false)
)

rf_clf.fit(X_train, y_train)

y_pred = rf_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

rf_clf.feature_importances_, accuracy
```

(array([0.00115382, 0.01762957, 0.00587829, 0.00355239, 0.00952323, 0.0069228, 0.02602209, 0.00846723, 0.03600714, 0.00132157, 0.00023384, 0.00654442, 0.0088773, 0.01622614, 0.03260653, 0.05072007, 0.00698333, 0.01197435, 0.04467999, 0.02283518, 0.03635831, 0.02445977, 0.04428716, 0.01066808, 0.06214612, 0.01101908, 0.0377339, 0.03571032, 0.03032146, 0.00961027, 0.04863299, 0.04818367, 0.01749102, 0.01277816, 0.02838997, 0.01608629, 0.0137547, 0.00544854, 0.02506656, 0.02957197, 0.00031868, 0.0005687, 0.02236622, 0.0034734, 0.04765376, 0.02672186, 0.01939419, 0.00231035]), 0.912962962962963)

Random Forests

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf_clf = RandomForestClassifier(
    n_estimators=100,      # Number of trees in the forest
    max_depth=5,          # Maximum depth of each tree
    max_leaf_nodes=15,    # Maximum leaf nodes each tree can have
    min_samples_leaf=2,   # Minimum samples required at each leaf node
    random_state=42,      # Ensures reproducibility
    max_features='sqrt',  # Number of features to consider for splits
    bootstrap=True        # Whether to use bagging (true) or pasting (false)
)

rf_clf.fit(X_train, y_train)

y_pred = rf_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

rf_clf.feature_importances_, accuracy
```

```
(array([0.00952323, 0.00115382, 0.01762957, 0.00587829, 0.00355239,
        0.02602209, 0.0069228 , 0.00846723, 0.03600714, 0.00132157,
        0.00023384, 0.00654442, 0.0088773 , 0.01622614,
        0.03260653, 0.05072007, 0.00698333, 0.03635831, 0.02445977,
        0.01197435, 0.04467999, 0.02283518, 0.03571032, 0.03032146,
        0.04428716, 0.06214612, 0.01101908, 0.0377339 , 0.01749102,
        0.01066808, 0.00961027, 0.04863299, 0.04818367, 0.01277816,
        0.02838997, 0.00544854, 0.02506656, 0.02957197,
        0.01608629, 0.0137547 , 0.0005687 , 0.02236622, 0.0034734 ,
        0.04765376, 0.02672186, 0.01939419, 0.00231035]),
0.912962962962963)
```

Parametri:

- **n_estimators**: numărul de arbori
- **parametri specifici arborilor de decizie**
- **max_features**: similar cu decision trees
 - dacă e 'sqrt', se iau $\sqrt{d}$ caracteristici
 - d e numărul total de caracteristici
- **bootstrap**: parametru specific bagging/pasting

Pentru regresie, se folosește clasa
RandomForestRegressor

Random Forests

Parametri:

- **n_estimators**: numărul de arbori
- **parametri specifici arborilor de decizie**
- **max_features**: similar cu decision trees
 - dacă e 'sqrt', se iau $\sqrt{d}$ caracteristici
 - d e numărul total de caracteristici
- **bootstrap**: parametru specific bagging/pasting

Pentru regresie, se folosește clasa RandomForestRegressor

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

rf_reg = RandomForestRegressor(
    n_estimators=100,      # Number of trees in the forest
    max_depth=5,          # Maximum depth of each tree
    max_leaf_nodes=15,    # Maximum number of leaf nodes
    min_samples_leaf=2,   # Minimum samples required at each leaf node
    random_state=42,      # Ensures reproducibility
    max_features='sqrt',  # Number of features to consider for splits
    bootstrap=True        # Whether to use bagging (true) or pasting (false)
)

rf_reg.fit(X_train, y_train)

y_pred = rf_reg.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

rf_reg.feature_importances_, mse
```

(array([0.03226298, 0.04254156, 0.05068989, 0.33950791, 0.00847184,
 0.00971082, 0.26727304, 0.01310673, 0.0036863 , 0.23274895])),
6052.0017162642025)

- **Până acum am antrenat modele în paralel pe diferite subseturi de date pentru a crește performanța generală**
- **O altă metodă de a crește performanța generală folosind mai multe modele:**
 - **Pornim de la un model de bază**
 - **Antrenăm un alt model pe greșelile primului model**
 - **Antrenăm un alt model pe greșelile celui de al doilea model**
 - **Continuăm procesul până ajungem la N modele**
- **Toate cele N modele vor fi folosite la predicție pe bază de votare**

Boosting

Boosting

Regulă de bază:

- Fiecare model (sau „învățăcel slab”) este antrenat secvențial, iar fiecare model se concentrează mai mult pe greșelile modelului anterior.
- După ce toate modelele sunt antrenate, predicțiile lor sunt combinate într-un mod ponderat. Modelele care au avut performanțe mai bune primesc o pondere mai mare în decizia finală, dar toate modelele fac parte din ansamblul final.

Două metode bine-cunoscute: AdaBoost și GradientBoosting.

AdaBoost

Cum funcționează?

- Prima dată, antrenăm un model pe date. Dacă acest model face greșeli, AdaBoost acordă o pondere mai mare punctelor de date care au fost clasificate greșit.
- Apoi, al doilea model este antrenat pe aceleași date, dar cu mai multă atenție pe greșelile primului model (puncte de date cu greutatea mai mari).
- Acest proces continuă, iar la final combinăm toate modelele antrenate, fiecare având o contribuție ponderată, pentru a face predicția finală.

AdaBoost

4
3C.



Codul sugerat poate face obiectul unei licențe | zhaowendao30/MySklearn | siddot/MNIST

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=3),
    n_estimators=200,
    learning_rate=0.5,
    random_state=42
)

ada_clf.fit(X_train, y_train)

y_pred = ada_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

accuracy
```



```
/usr/local/lib/python3.10/dist-packages/sklearn/ensemble/_weight_boosting.py:527: FutureWarning
  warnings.warn(
0.9444444444444444
```


AdaBoost

```
4  Codul sugerat poate face obiectul unei licențe | zhaowendao30/MySklearn | siddot/MNIST
bc.

from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

digits = load_digits()
X, y = digits.data, digits.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ada_clf = AdaBoostClassifier(
    DecisionTreeClassifier(max_depth=3),
    n_estimators=200,
    learning_rate=0.5,
    random_state=42
)

ada_clf.fit(X_train, y_train)

y_pred = ada_clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

accuracy

/usr/local/lib/python3.10/dist-packages/sklearn/ensemble/_weight_boosting.py:527: FutureWarning
warnings.warn(
0.9444444444444444
```

Parametri:

- modelul folosit în a realiza ansamblul
- numărul de modele din ansamblu
- rata de contribuție a fiecărui model nou

Dacă learning_rate este mare, fiecare model nou are un impact mai puternic asupra predicției finale.

Dacă learning_rate este mic, fiecare model nou corectează erorile mai încet și cu mai multă grijă.

AdaBoost (la regresie)

```
from sklearn.ensemble import AdaBoostRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_regression
from sklearn.metrics import mean_squared_error

X, y = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ada_reg = AdaBoostRegressor(
    DecisionTreeRegressor(max_depth=3),
    n_estimators=200,
    learning_rate=0.5,
    random_state=42
)

ada_reg.fit(X_train, y_train)

y_pred = ada_reg.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

mse
```

3823.671426813229

Parametri:

- **modelul folosit în a realiza ansamblul**
- **numărul de modele din ansamblu**
- **rata de contribuție a fiecărui model nou**

Dacă learning_rate este mare, fiecare model nou are un impact mai puternic asupra predicției finale.

Dacă learning_rate este mic, fiecare model nou corectează erorile mai încet și cu mai multă grijă.

Gradient Boosting

Cum funcționează?

- La fiecare pas, Gradient Boosting creează un nou model care corectează rezidualii (diferențele între predicția modelului anterior și valoarea reală).
- Fiecare nou model nu încearcă să clasifice datele, ci încearcă să prezică erorile modelului anterior, astfel încât, treptat, predicțiile să fie mai precise.
- După ce toate modelele sunt construite, rezultatele lor sunt combinate pentru a obține predicția finală.

Gradient Boosting

- Începem cu un model obișnuit:

```
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3*X[:, 0]**2 + 0.05 * np.random.randn(100)

tree_reg1 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg1.fit(X, y)
```

- Antrenăm un al doilea model pe erorile de la primul:

```
y2 = y - tree_reg1.predict(X)
print(y2)
tree_reg2 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg2.fit(X, y2)
```

- Antrenăm un al treilea model pe erorile de la al doilea:

```
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeRegressor(max_depth=2, random_state=42)
tree_reg3.fit(X, y3)
```

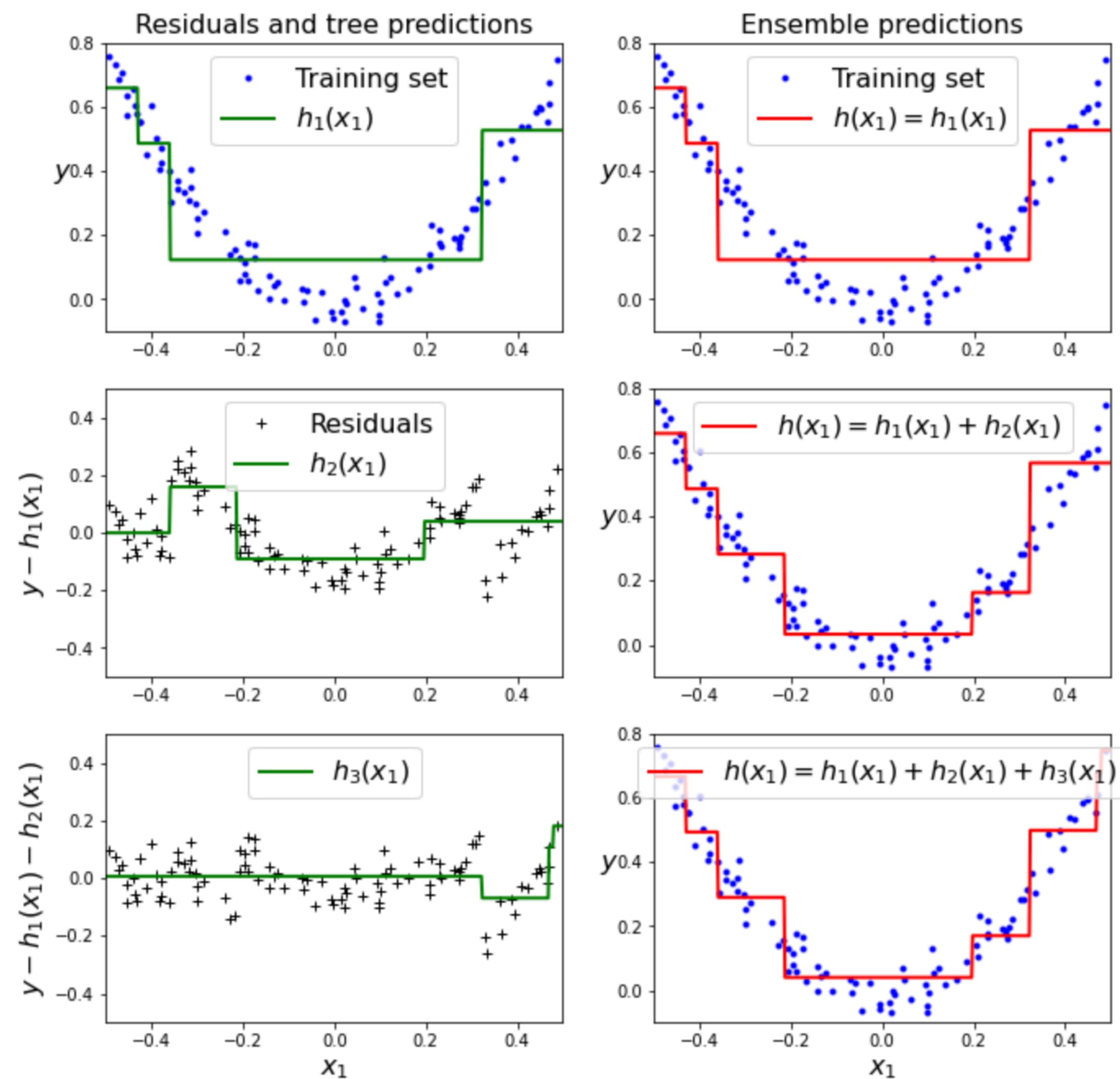
- Combinăm rezultatele celor 3 modele

```
X_new = np.array([[0.8]])

y_pred = sum(tree.predict(X_new) for tree in (tree_reg1, tree_reg2, tree_reg3))
y_pred

array([0.75026781])
```


Gradient Boosting



GradientBoosting

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0]**2 + 0.05 * np.random.randn(100)

X_train, X_val, y_train, y_val = train_test_split(X, y, random_state=49)

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=200, learning_rate=0.1, random_state=42)
gbrt.fit(X_train, y_train)

y_pred = gbrt.predict(X_val)
mse = mean_squared_error(y_val, y_pred)

mse
```

0.003173164587599602

Există și GradientBoostingClassifier pentru clasificare.

GradientBoosting

```
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0]**2 + 0.05 * np.random.randn(100)

X_train, X_val, y_train, y_val = train_test_split(X, y, random_state=49)

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=200, learning_rate=0.1, random_state=42)
gbrt.fit(X_train, y_train)

y_pred = gbrt.predict(X_val)
mse = mean_squared_error(y_val, y_pred)

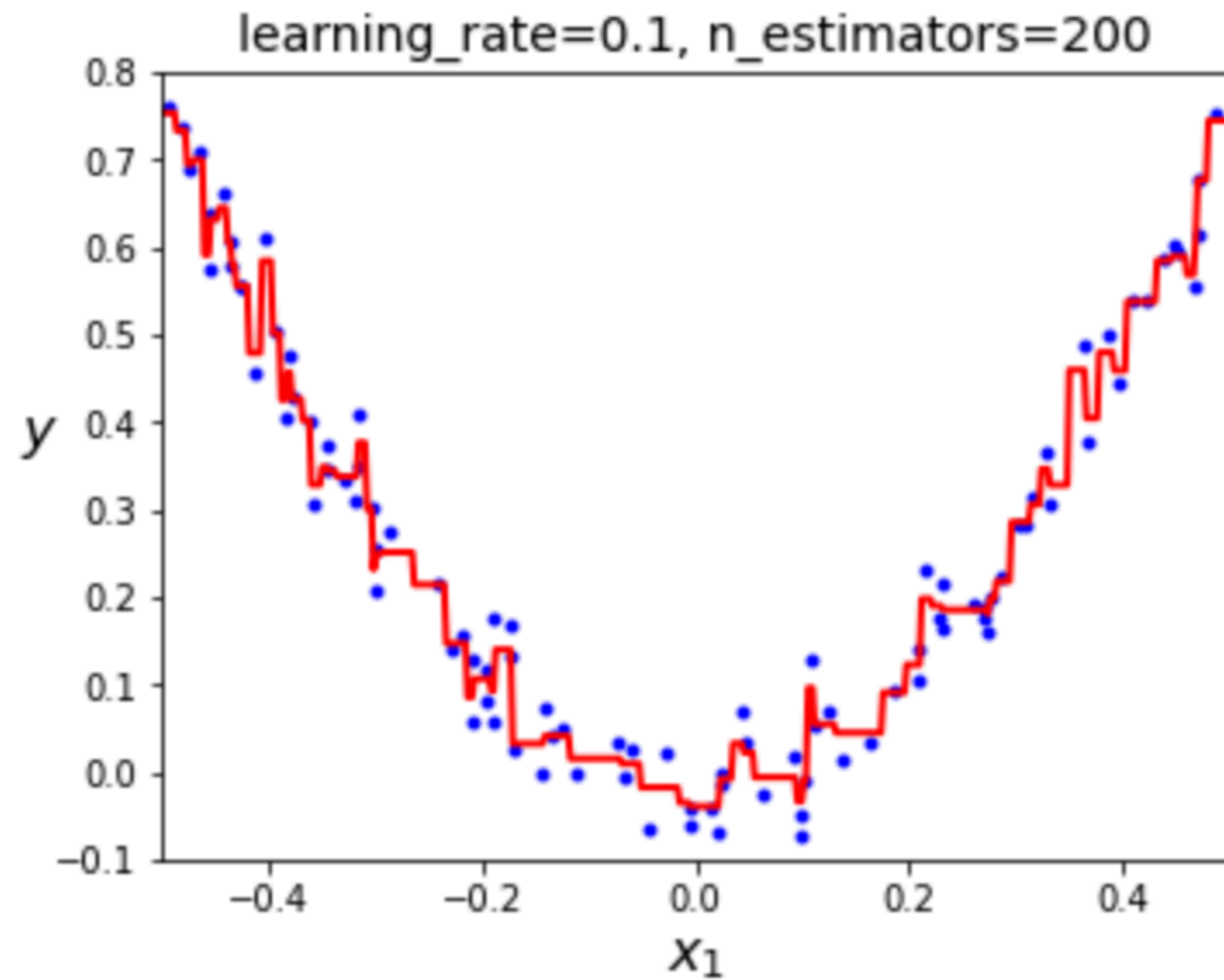
mse
```

0.003173164587599602

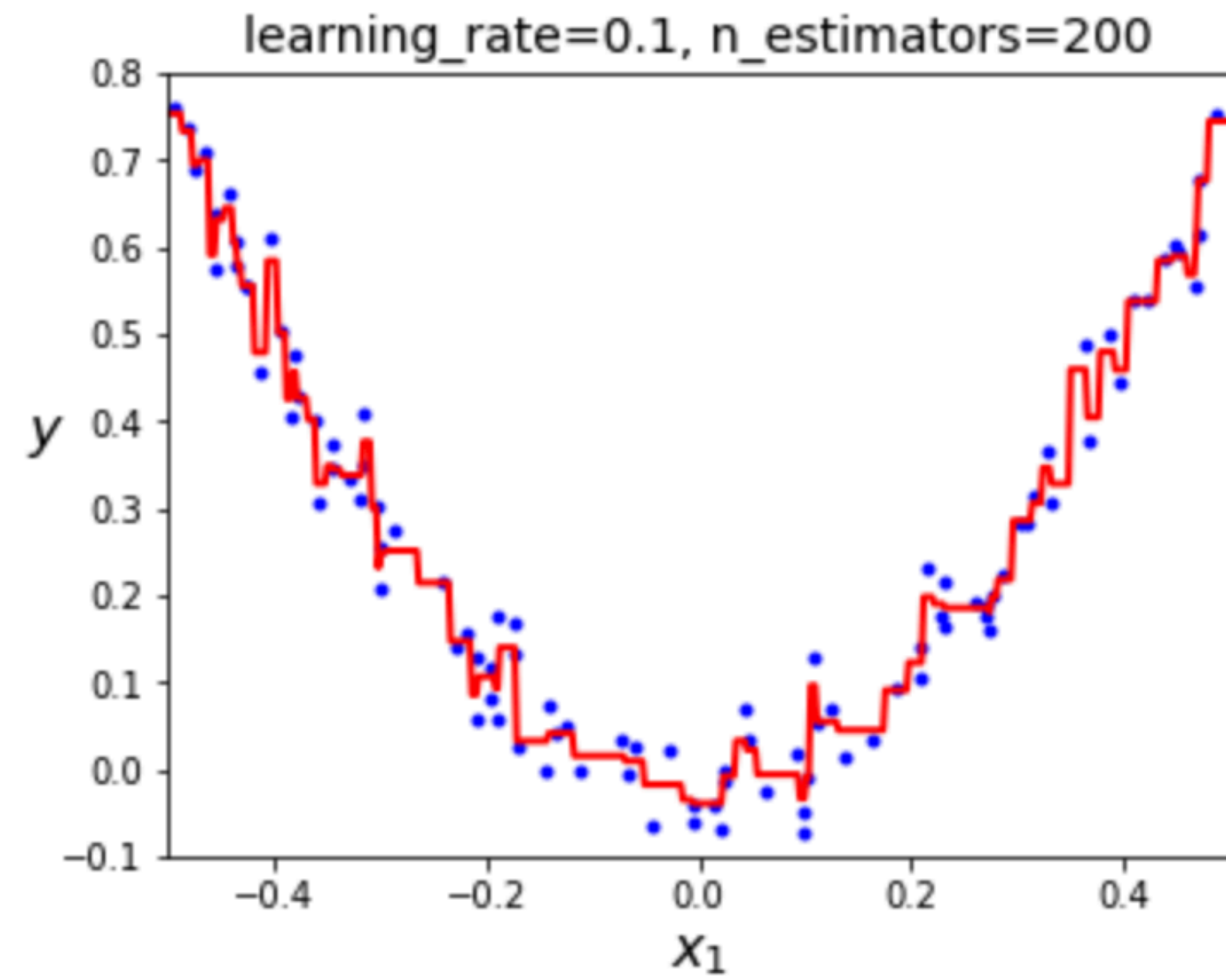
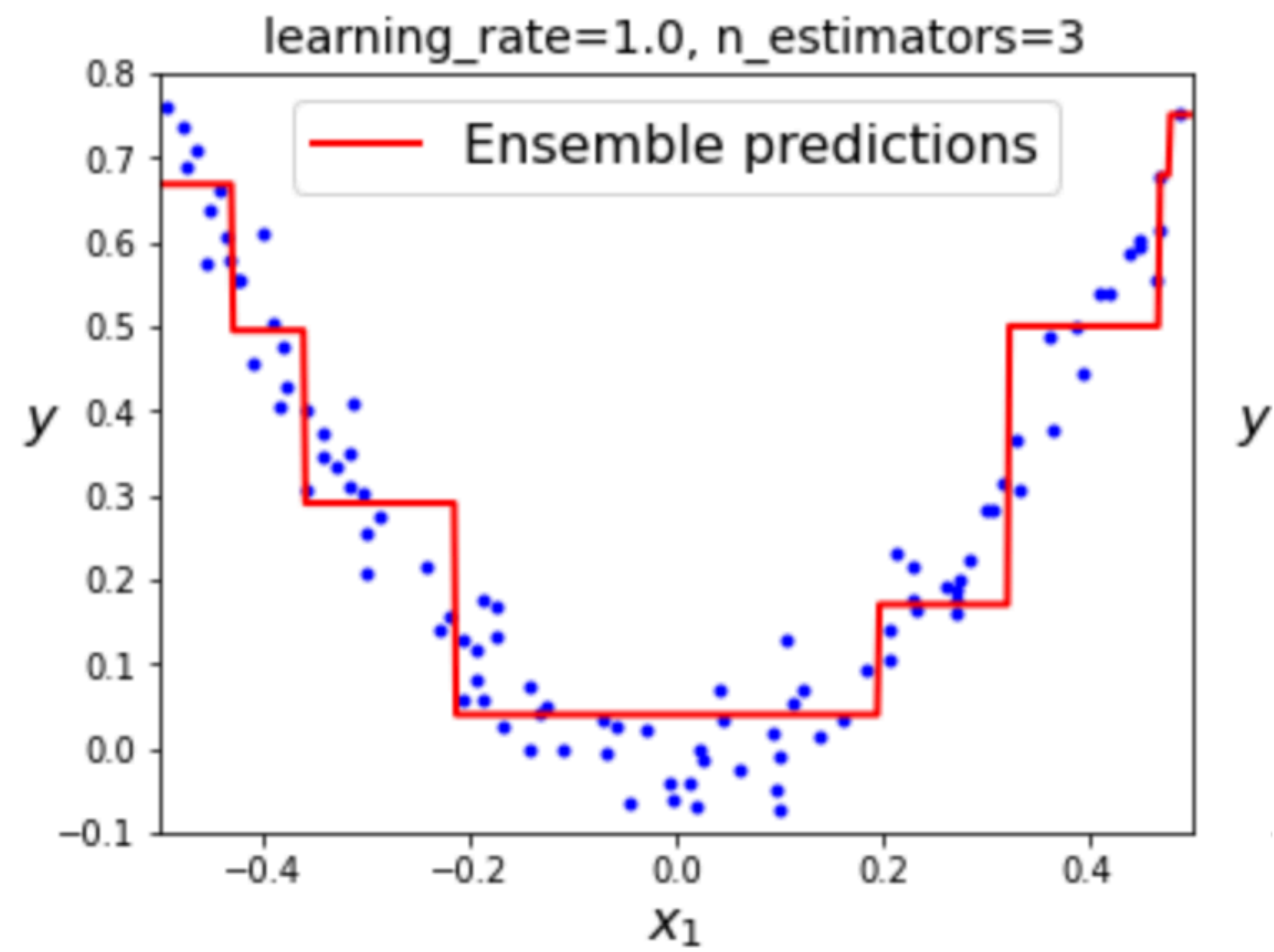
Dacă learning_rate este mare, fiecare model nou are un impact mai puternic asupra predicției finale.

Dacă learning_rate este mic, fiecare model nou corectează erorile mai încet și cu mai multă grijă.

GradientBoosting



GradientBoosting



GradientBoosting + Early Stopping



Codul sugerat poate face obiectul unei licențe | [deel-ai/puncc](#) | [PacktPublishing/Comet-for-Data-Science](#)

```
np.random.seed(42)
```

```
X = np.random.rand(100, 1) - 0.5
```

```
y = 3 * X[:, 0]**2 + 0.05 * np.random.randn(100)
```

```
X_train, X_val, y_train, y_val = train_test_split(X, y, random_state=49)
```

```
gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=200, learning_rate=0.1, random_state=42)  
gbrt.fit(X_train, y_train)
```

```
y_pred = gbrt.predict(X_val)
```

```
mse = mean_squared_error(y_val, y_pred)
```

```
errors = [mean_squared_error(y_val, y_pred) for y_pred in gbrt.staged_predict(X_val)]
```

```
bst_n_estimators = np.argmin(errors) + 1
```

```
gbrt_best = GradientBoostingRegressor(max_depth=2, n_estimators=bst_n_estimators, learning_rate=0.1, random_state=42)  
gbrt_best.fit(X_train, y_train)
```

```
y_pred_best = gbrt_best.predict(X_val)
```

```
mse_best = mean_squared_error(y_val, y_pred_best)
```

```
mse, bst_n_estimators, mse_best
```

La fel ca înainte



```
(0.003173164587599602, 56, 0.002712853325235463)
```


GradientBoosting + Early Stopping

```
errors = [mean_squared_error(y_val, y_pred) for y_pred in gbrt.staged_predict(X_val)]  
bst_n_estimators = np.argmin(errors) + 1
```

Funcția `staged_predict` ne permite să calculăm prezicerile în stagii intermediare

De exemplu, va face predicția doar cu primul estimator, după doar cu primul două, după cu primele trei, și tot așa, până ajunge să facă predicția cu toate.

Ne va oferi prezicerile la aceste stagii intermediare pentru a putea selecta numărul optim de estimatori.

GradientBoosting + Early Stopping

```
errors = [mean_squared_error(y_val, y_pred) for y_pred in gbrt.staged_predict(X_val)]  
bst_n_estimators = np.argmin(errors) + 1
```

Practic, aici calculăm eroarea pentru 1 estimator, 2 estimatori, până la N estimatori

Obținem o listă de erori

Cu np.argmin calculăm indicele din listă unde se află eroarea cea mai mică

Adunăm un 1 deoarece indexarea în liste începe de la 0

GradientBoosting + Early Stopping



Codul sugerat poate face obiectul unei licențe | [deel-ai/puncc](#) | [PacktPublishing/Comet-for-Data-Science](#)

```
np.random.seed(42)
X = np.random.rand(100, 1) - 0.5
y = 3 * X[:, 0]**2 + 0.05 * np.random.randn(100)

X_train, X_val, y_train, y_val = train_test_split(X, y, random_state=49)

gbrt = GradientBoostingRegressor(max_depth=2, n_estimators=200, learning_rate=0.1, random_state=42)
gbrt.fit(X_train, y_train)

y_pred = gbrt.predict(X_val)
mse = mean_squared_error(y_val, y_pred)

errors = [mean_squared_error(y_val, y_pred) for y_pred in gbrt.staged_predict(X_val)]

bst_n_estimators = np.argmin(errors) + 1

gbrt_best = GradientBoostingRegressor(max_depth=2, n_estimators=bst_n_estimators, learning_rate=0.1, random_state=42)
gbrt_best.fit(X_train, y_train)

y_pred_best = gbrt_best.predict(X_val)
mse_best = mean_squared_error(y_val, y_pred_best)

mse, bst_n_estimators, mse_best
```



```
(0.003173164587599602, 56, 0.002712853325235463)
```


That's it for today!
Now let's do some exercises.

That's it for today!

Now let's do some exercises.

Laboratory 4 – Exercises

1. Use the following regression dataset:

```
X = np.random.rand(350,1)
y = (X + 1.2)^3 - 10 + np.random.randn(350,1)/10,
```

in order to perform a regression task. Build a regression decision tree with `max_depth=3` and `min_samples_leaf=5`. Visualize the trained decision tree and plot the regression predictions.

2. Use the `digits` dataset to perform a classification task using bagging. Set the number of estimators to 200. The test split should be 15% of the initial dataset. Print the accuracy of the model.

3. Use the following regression dataset:

```
X = np.random.rand(160,1) - 1
y = 2*(X[:, 0])^2 + 0.2*np.random.randn(160),
```

in order to perform gradient boosting. The `GradientBoostingRegressor` class has the hyperparameters: `max_depth=2` and `n_estimators=100`. The test split is 25% of the initial dataset. Use early stopping to find the optimal number of trees and plot:

- the model's predictions with 100 estimators;
- the model's predictions with the optimal number of trees.